ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

# 1. ВВЕДЕНИЕ В ТЕОРИЮ АЛГОРИТМОВ

## 1.1. Основные понятия, формальные определения алгоритма, его свойства

Понятие алгоритма стихийно формировалось с древнейших времен. Современный человек понимает под алгоритмом четкую систему инструкций о выполнении в определенном порядке некоторых действий для решения всех задач какого-то данного класса.

Многочисленные и разнообразные алгоритмы окружают нас буквально во всех сферах жизни и деятельности. Многие наши действия доведены до бессознательного автоматизма, мы порой и не осознаем, что они регламентированы определенным алгоритмом – четкой системой инструкций. Например, наши действия при входе в магазин «Универсам» (сдать свою сумку, получить корзину с номером, пройти в торговый зал, заполнить корзину продуктами, оплатить покупку в кассе, предъявить чек контролеру, взять свою сумку, переложить в нее продукты, сдать корзину, покинуть магазин). Второй пример – приготовление манной каши (500 мл молока довести до кипения, при тщательном помешивании засыпать 100 г манной крупы, при помешивании довести до кипения и варить 10 минут). Автоматизм выполнения этих и многих других действий не позволяет нам осознавать их алгоритмическую сущность.

Но есть немало таких действий, выполняя которые, мы тщательно следуем той или иной инструкции. Это главным образом непривычные действия, профессионально не свойственные нам. Например, если вы фотографируете один-два раза в год, то, купив проявитель для пленки, будете весьма тщательно следовать инструкции (алгоритму) по его приготовлению: «Содержимое большого пакета растворить в 350 мл воды при температуре 18– 20 °С. Там же растворить содержимое малого пакета. Объем раствора довести до 500 мл. Раствор профильтровать. Проявлять 3–4 роликовых фотопленки». Второй пример: если вы никогда раньше не пекли торт, то, получив рецепт (алгоритм) его приготовления, постараетесь выполнить в указанной последовательности все его предписания.

Большое количество алгоритмов встречается при изучении математики буквально с первых классов школы. Это прежде всего алгоритмы выполнения четырех арифметических действий над различными числами – натуральными, целыми, дробными, комплексными. Вот пример такого алгоритма: «Чтобы из одной десятичной дроби вычесть другую, надо: 1) уравнять число знаков после запятой в уменьшаемом и вычитаемом; 2) записать вычитаемое под уменьшаемым так, чтобы запятая оказалась под запятой; 3) произвести вычитание так, как вычитают натуральные числа; 4) поставить в полученной разности запятую под запятыми в уменьшаемом и вычитаемом».

---

Вот пример алгоритма сложения приближенных чисел. Найти сумму чисел $p$, $q$ и $s$, где $p \approx 3{,}1416$, $q \approx 2{,}718$ и $s \approx 7{,}45$.

1. Выделим слагаемое с наименьшим числом десятичных знаков. Таким слагаемым является число 7,45 (два десятичных знака).

2. Округлим остальные слагаемые, оставляя в них столько десятичных знаков, сколько их имеется в выделенном слагаемом: $3{,}1416 \approx 3{,}14$; $2{,}718 \approx 2{,}72$.

3. Выполним сложение приближенных значений чисел: $3{,}14 + 2{,}72 + 7{,}45 = 13{,}31$.

Итак, $p + q + 5 \approx 13{,}31$.

Немало алгоритмов в геометрии: алгоритмы геометрических построений с помощью циркуля и линейки (деление пополам отрезка и угла, опускание и восстановление перпендикуляров, проведение параллельных прямых), алгоритмы вычисления площадей и объемов различных геометрических фигур и тел.

При изучении математики в вузе были освоены процедуры вычисления наибольшего общего делителя двух натуральных чисел (алгоритм Евклида), определителей различных порядков, рангов матриц с рациональными элементами, интегралов от рациональных функций, приближенных значений корней уравнений и систем и т.д. Все эти процедуры являются не чем иным, как алгоритмами.

Одним словом, алгоритмы широко распространены как в практике, так и в науке и требуют более внимательного к себе отношения и тщательного изучения методами математической науки.

Алгоритм – это точное предписание, которое задает вычислительный {алгоритмический) процесс, начинающийся с произвольного исходного данного (из некоторой совокупности возможных для данного алгоритма исходных данных) и направленный на получение полностью определяемого этим исходным данным результата [11]. Алгоритм описывает порядок действий исполнителя для решения некоторой задачи. В наше время в качестве исполнителя алгоритма чаще всего выступает компьютер. В этом случае алгоритм называют также компьютерной программой. Однако исполнителем может быть и человек. Так. именно для человека формулируются известные из начальной школы правила (т. е. алгоритмы) сложения, вычитания, умножения и деления чисел столбиком. В этих алгоритмах возможными исходными данными служат упорядоченные нары натуральных чисел, записанных в десятичной системе, а возможными результатами – натуральные числа. Именно с правилами арифметических действий связано само появление термина «алгоритм». Его происхождение связывают с именем среднеазиатского ученого аль-Хорезми (полное имя Абу Абдалла Мухаммед бен Муса аль-Маджуси аль-Хорезми), жившего в IX в., описавшего изобретенную в Индии позиционную десятичную систему счисления и сформулировавшего правила вычислений в этой системе. Именно благодаря латинскому переводу трактата аль-Хорезми (XII в.) позиционная система счисления стала известной в Европе, а правила счета в этой системе получили название алгоритмов по латинской транскрипции имени автора трактата. В широком

смысле, к алгоритмам можно отнести всякое точное предписание (например, подробный рецепт приготовления некоторого блюда или инструкцию по сборке предмета мебели из деталей). Однако мы будем рассматривать лишь алгоритмы, направленные на решение математических задач.

Роль алгоритмов в математике определяется тем, что на протяжении всей истории ее развития возникают и решаются задачи. явно или косвенно связанные с понятием алгоритма. Эго происходит всякий раз, когда речь идет об умении решать задачу «в общем виде». Например, говоря об умении человека извлекать квадратный корень, имеют в виду, что он владеет некоторым единообразным приемом (т. с. алгоритмом) извлечения квадратного корня. Понятие решения задачи «в общем виде» уточняется в математике при помощи понятия массовой алгоритмической проблемы, состоящей в требовании найти единый алгоритм для решения некоторой серии отдельных, единичных проблем. К ним относятся, например, возникающие в алгебре проблемы численного решения уравнений того или иного типа, проверки алгебраических равенств и т. п. В настоящее время построение алгоритмов для численного решения математических задач особенно актуально в связи с широким использованием вычислительной техники.

Положительное решение алгоритмической проблемы, т. е. построение соответствующего алгоритма, не требует специального анализа понятия алгоритма. В каждом конкретном случае обычно бывает ясно, является ли предложенное общее правило решения данной серии задач алгоритмом. Так, например, в математике известны процесс нахождения наибольшего общего делителя двух целых чисел (алгоритм Евклида), алгоритмы разложения натурального числа на простые множители, извлечения квадратного корня из натурального числа, решения системы линейных уравнений методом Гаусса и т.д. Однако со временем математика столкнулась с такими алгоритмическими проблемами, которые не поддавались решению. Одной из наиболее известных (это не значит, что наиболее важных) таких проблем является десятая проблема Гильберта, состоящая в требовании найти алгоритм для распознавания разрешимости в целых числах диофантовых уравнений. Трудности решения этой и ряда других алгоритмических проблем постепенно привели к осознанию того, что искомый алгоритм. возможно, вообще не существует. Но для строгого математического доказательства невозможности алгоритма для решения той или иной массовой проблемы требуется строгое математическое определение понятия алгоритма и исследование общих свойств алгоритмов. Этим и занимается раздел современной математики, который называется теория алгоритмов.

Математическое понятие алгоритма стало предметом самостоятельного изучения лишь в XX в. В настоящее время известны различные математические уточнения понятия алгоритма и связанного с ним понятия вычислимости. Все они оказались в некотором смысле эквивалентными и отражают общие интуитивные представления об алгоритмах. Эти представления можно сформулировать в виде следующих общих свойств алгоритмов.

1. Дискретность. Выполнение алгоритма состоит из отдельных шагов. Каждый шаг обязательно завершается. Если выполнение алгоритма никогда не закончится, это означает, что алгоритм совершает бесконечное число шагов. В случае компьютерной программы шагом работы считается выполнение одной команды.

2. Элементарность шагов. Выполнение каждого шага работы алгоритма должно быть простым и не требовать какой-либо изобретательности от исполнителя. Это свойство наглядно проявляется при выполнении алгоритма компьютером, где способ исполнения команд изначально заложен в операционную систему.

3. Детерминированность. Результат выполнения каждою шага алгоритма однозначно определяется программой и результатами выполнения предшествующих шагов алгоритма.

4. Направленность. Должно быть указано, что надо считать результатом алгоритма.

5. Массовость. Исходные данные для алгоритма могут выбираться из заранее фиксированного множества возможных исходных данных. Обычно это множество бесконечно.

6. Связанность. На каждом следующем шаге используются результаты предыдущих.

7. Конечность. Алгоритм заканчивает работу после конечного числа шагов.

Алгоритм также можно рассматривать как инструмент, предназначенный для решения корректно поставленной вычислительной задачи (computational problem) [17]. В постановке задачи в общих чертах задаются отношения между входом и выходом. В алгоритме описывается конкретная вычислительная процедура, с помощью которой удается добиться выполнения указанных отношений.

Например, может понадобиться выполнить сортировку последовательности чисел в неубывающем порядке. Эта задача часто возникает на практике и служит благодатной почвой для ознакомления на ее примере со многими стандартными методами разработки и анализа алгоритмов. Задача сортировки (sorting problem) формально определяется следующим образом.

Вход: последовательность из $n$ чисел.

Выход: перестановка (изменение порядка) входной последовательности.

Например, если на вход подастся последовательность 31, 41, 33 то вывод алгоритма сортировки должен быть таким: 41, 31, 33. Подобная входная последовательность называется экземпляром (instance) задачи сортировки. Вообще говоря, экземпляр задачи состоит из ввода (удовлетворяющего всем ограничениям, наложенным при постановке задачи), необходимою для решения задачи.

В информатике сортировка является основополагающей операцией (во многих программах она используется в качестве промежуточного шага), в результате чего появилось большое количество хороших алгоритмов сортировки. Выбор наиболее подходящего алгоритма зависит от многих факторов, в том числе от количества

сортируемых элементов, от их порядка во входной последовательности, от возможных ограничений, накладываемых на члены последовательности, а также от того, какое устройство используется для хранения последовательности: основная память, магнитные диски или накопители на магнитных лентах.

Говорят, что алгоритм *корректен* (correct), если для каждого ввода результатом его работы является корректный вывод. Мы говорим, что корректный алгоритм решает данную вычислительную задачу. Если алгоритм некорректный, то для некоторых вводов он может вообще не завершить свою работу или выдать ответ, отличный от ожидаемого. Правда, некорректные алгоритмы иногда могут оказаться полезными, если в них есть возможность контролировать частоту возникновения ошибок, например алгоритмы определения простых чисел, намного превышающих единицу. Тем не менее, обычно мы заинтересованы только в корректных алгоритмах.

Алгоритм может быть задан на естественном языке, в виде компьютерной программы или даже воплощен в аппаратном обеспечении. Единственное требование – его спецификация должна предоставлять точное описание вычислительной процедуры, которую требуется выполнить.

Предположим, быстродействие компьютера и объем его памяти можно увеличивать до бесконечности. Была бы в таком случае необходимость в изучении алгоритмов? Была бы. но только для того, чтобы продемонстрировать, что метод решения имеет конечное время работы и что он дает правильный ответ.

Если бы компьютеры были неограниченно быстрыми, подошел бы любой корректный метод решения задачи. Возможно, вы бы предпочли, чтобы реализация решения была выдержана в хороших традициях программирования (т.е. качественно разработана и аккуратно -занесена в документацию), но чаше всего выбирался бы метод, который лепте всего реализовать.

Конечно же, сегодня есть весьма производительные компьютеры, но их быстродействие не может быть бесконечно большим. Память гоже дешевеет, но она не может быть бесплатной. Таким образом, время вычисления – это такой же ограниченный ресурс, как и объем необходимой памяти. Этими ресурсами следует распоряжаться разумно, чему и способствует применение алгоритмов, эффективных в плане расходов времени и памяти.

Алгоритмы, разработанные для решения одной и то же задачи, часто очень сильно различаются по эффективности. Эти различия могут быть намного значительнее, чем те, что вызваны применением неодинакового аппаратного и программного обеспечения.

---

В качестве примера можно привести два алгоритма сортировки. Для выполнения первого из них, известного как сортировка вставкой, требуется время, которое оценивается как $c_1 n^2$, где $n$ – количество сортируемых элементов, а $c_1$ – константа, не зависящая от $n$. Таким образом, время работы этого алгоритма приблизительно пропорционально $n^2$. Для выполнения

второго алгоритма, сортировки слиянием, требуется время, приблизительно равное $c_2 n \log_2 n$, а $c_2$ – некоторая другая константа, не зависящая от $n$. Обычно константа метода вставок меньше константы метода слияния, т.е. $c_1 < c_2$. Постоянные множители намного меньше влияют на время работы алгоритма, чем множители, зависящие от $n$. Для двух приведенных методов последние относятся как $\log_2 n$ к $n$. Для небольшого количества сортируемых элементов сортировка включением обычно работает быстрее, однако когда $n$ становится достаточно большим, все заметнее проявляется преимущество сортировки слиянием, возникающее благодаря тому, что для больших $n$ незначительная величина $\log_2 n$ по сравнению с п полностью компенсирует разницу величин постоянных множителей. Не имеет значения, во сколько раз константа $c_1$ меньше, чем $c_2$. С ростом количества сортируемых элементов обязательно будет достигнут переломный момент, когда сортировка слиянием окажется более производительной.

В качестве примера рассмотрим два компьютера – А и Б. Компьютер А более быстрый, и на нем работает алгоритм сортировки вставкой, а компьютер Б более медленный, и на нем работает алгоритм сортировки методом слияния. Оба компьютера должны выполнить сортировку множества, состоящего из миллиона чисел. Предположим, что компьютер А выполняет миллиард инструкций в секунду. а компьютер Б – лишь десять миллионов. Таким образом, компьютер А работает в 100 раз быстрее, чем компьютер Б. Чтобы различие стало еще большим, предположим, что код для метода вставок (т.е. для компьютера А) написан самым лучшим в мире программистом с использованием команд процессора, и для сортировки $n$ чисел надо выполнить $2n^2$ команд (т.е. $c_1 = 2$). Сортировка же методом слияния (на компьютере Б) реализована программистом-середнячком с помощью языка высокого уровня. При этом компилятор оказался не слишком эффективным, и в результате получился кол, требующий выполнения $50n \log_2 n$ команд (т.е. $c_2 = 50$). Для сортировки миллиона чисел компьютеру А понадобится

$$\frac{2 \cdot (10^6)^2}{10^9} = 2000 \text{ с},$$

а компьютеру Б –

$$\frac{50 \cdot 10^6 \cdot \log_2 10^6}{10^7} \approx 100 \text{ с}.$$

Как видите, использование кода, время работы которого возрастает медленнее, даже при плохом компиляторе на более медленном компьютере требует на порядок меньше процессорного времени! Если же нужно выполнить сортировку К) миллионов чисел, то преимущество метода слияния становится еще более очевидным: если для сортировки вставкой потребуется приблизительно 2,3 дня, то для сортировки слиянием – меньше 20 минут.

Общее правило таково: чем больше количество сортируемых элементов, тем заметнее преимущество сортировки слиянием.

Приведенный выше пример демонстрирует, что алгоритмы, как и аппаратное обеспечение компьютера, представляют собой технологию. Общая производительность системы настолько же зависит от эффективности алгоритма, как и от мощности применяющегося аппаратного обеспечения. В области разработки алгоритмов происходит такое же быстрое развитие, как и в других компьютерных технологиях.

Возникает вопрос, действительно ли так важны алгоритмы, работающие на современных компьютерах, если и так достигнуты выдающиеся успехи в других областях высоких технологий, таких как: аппаратное обеспечение с высокими тактовыми частотами, конвейерной обработкой и суперскалярной архитектурой; легкодоступные, интуитивно понятные графические интерфейсы (GUI); объектно-ориентированные системы; локальные и глобальные сети.

Ответ – да, безусловно. Несмотря на то, что иногда встречаются приложения, которые не требуют алгоритмического наполнения (например, некоторые простые Web-приложения), для большинства приложений определенное алгоритмическое наполнение необходимо. Например, рассмотрим Web-службу, определяющую, как добраться из одного места в другое (во время написания книги существовало несколько таких служб). В основе се реализации лежит высокопроизводительное аппаратное обеспечение, графический интерфейс пользователя, глобальная сеть и. возможно, объектно-ориентированный подход. Кроме того, для определенных операций, выполняемых данной Web-службой, необходимо использование алгоритмов – например, таких как вычисление квадратных корней (что может потребоваться для определения кратчайшего пути), визуализации карт и интерполяции адресов.

Более того, даже приложение, не требующее алгоритмического наполнения на высоком уровне, сильно зависит от алгоритмов. Ведь известно, что работа приложения зависит от производительности аппаратного обеспечения, а при его разработке применяются разнообразные алгоритмы. Все мы также знаем, что приложение тесно связано с графическим интерфейсом пользователя, а для разработки любого графического интерфейса пользователя требуются алгоритмы. Вспомним приложения, работающие в сети. Чтобы они могли функционировать, необходимо осуществлять маршрутизацию, которая, как уже говорилось, основана на ряде алгоритмов. Чаще всего приложения составляются на языке, отличном от машинного. Их код обрабатывается компилятором или интерпретатором, которые интенсивно используют различные алгоритмы. И таким примерам нет числа.

Кроме того, ввиду постоянного роста вычислительных возможностей компьютеров, они применяются для решения все более сложных задач. Как мы уже убедились на примере сравнительного анализа двух методов сортировки, с ростом

сложности решаемой задачи различия в эффективности алгоритмов проявляются все значительнее.

Знание основных алгоритмов и методов их разработки – одна из характеристик, отличающих умелого программиста от новичка. Располагая современными компьютерными технологиями, некоторые задачи можно решить и без основательного знания алгоритмов, однако знания в этой области позволяют достичь намного большего.

Интуитивное понятие алгоритма обладает целым рядом недостатков [18]. Очевидно, что такие понятия, использованные при описании общих свойств алгоритмов, как ясность, элементарность шагов и т.п., сами нуждаются в уточнении. Очевидно, что их словесные определения будут содержать новые понятия, которые снова потребуют уточнения и т.д. Как уже отмечалось, начиная с 30-х годов, было предложено несколько уточнений понятия алгоритма. Считается, что все они достаточно полно отражают основные черты интуитивного понятия алгоритма. Действительно, все формальные определения алгоритма в некотором смысле эквивалентны друг другу. Поэтому в теории алгоритмов применяется другой подход: выбирается конечный набор исходных объектов, которые объявляются элементарными и конечный набор способов построения их них новых объектов. Этот метод был уже использован в теории множеств и получил название конструктивного подхода.

Алгоритмические модели, которые претендуют на право считаться формализацией понятия «алгоритм», должны быть универсальными, т.е. допускать описание любых алгоритмов.

Можно выделить три основных типа универсальных алгоритмических моделей, различающихся исходными эвристическими соображениями относительно того, что такое алгоритм. Первый тип связывает понятие алгоритма с наиболее традиционными понятиями математики – вычислениями и числовыми функциями. Наиболее развитая и изученная модель этого типа – рекурсивные функции – является исторически первой формализацией понятия алгоритма.

Второй тип модели связан с развитием вычислительной техники и основан на представлении об алгоритме как о некотором детерминированном устройстве, способном выполнять в каждый отдельный дискретный момент времени весьма примитивные операции. Такое представление не оставляет сомнений в однозначности алгоритма и элементарности его шагов. Кроме того, эвристика этой модели близка к ЭВМ и, следовательно, к инженерной интуиции. Основной теоретической моделью этого типа является, как уже отмечалось ранее, созданная в 30-х годах концепция машины Тьюринга. Именно машина Тьюринга явилась моделью современной ЭВМ и способствовала развитию современной вычислительной техники.

Третий тип алгоритмических моделей – это преобразование слов в произвольных алфавитах, в которых элементарными операциями являются

подстановки, т.е. замены части слова (подслова) другим словом. Преимущества этого типа моделей заключаются в максимальной абстрактности и возможности применить понятие алгоритма к объектам произвольной, не обязательно числовой природы. Примерами моделей этого типа являются канонические системы Поста и нормальные алгоритмы Маркова. При этом общность формализации в конкретной модели не теряется и доказывается сводимость одних моделей к другим, т.е. показывается, что всякий алгоритм, описанный средствами одной модели, может быть описан средствами другой.

Благодаря взаимной сводимости моделей в общей теории алгоритмов удалось выработать инвариантную по отношению к моделям систему понятий, позволяющую говорить о свойствах алгоритмов независимо от того, какая формализация алгоритма выбрана. Эта система понятий основана на понятии вычислимой функции, т.е. функции, для вычисления которой существует алгоритм.

Существуют следующие способы записи алгоритмов: на естественном языке, на псевдокоде, графические в виде схем-программ, на языках программирования.

Достоинством записи алгоритма на естественном языке является доступность для понимания его любым человеком.

Недостатки записи алгоритмов на естественном языке состоят в громоздкости записи, ненаглядности, неточности и многозначности.

Псевдокод – это язык со свободным синтаксисом, близким к естественному языку или языкам программирования, обладающий свойствами: краткости записи алгоритмов; понятности для ограниченного круга людей; естественности и простоты описания логики решения задачи; возможности использования собственных символьных конструкций; возможности представления алгоритма с произвольной степенью детализации.

Графическая запись алгоритмов в виде блок-схем (схем алгоритмов, схем программ) характеризуется следующими свойствами: использованием графических символов, математических записей и записей на естественном языке; наглядностью; пониманием записи алгоритма любым человеком, знакомым с алгоритмами; возможностью представления алгоритма с произвольной степенью детализации; использованием простых правил описания последовательностей действий.

Для записи алгоритмов могут использоваться специальные искусственные языки, описания на которых могут исполняться компьютером. Такое предписание для компьютера рассматривается в качестве программы. Программа – это предписание компьютеру на языке программирования, позволяющее решать требуемую задачу.

Языком программирования называется (формальный) язык, предназначенный для записи программ, исполняемых на компьютере. Синтаксис языка программирования – это набор правил построения конструкций языка. Семантика языка программирования – это совокупность значений (смысл) всех конструкций

языка. Алгоритмическим языком программирования называется язык программирования, предназначенный для записи алгоритмов.

Запись алгоритмов на языке программирования характеризуется следующими свойствами: точность передачи содержания вычислений; понимание человеком-программистом; возможность применения компьютера для исполнения программы; формальность выполнения алгоритма; громоздкость записи; ненаглядность.

## 1.2. Классификации алгоритмов

Существует большое многообразие классификации алгоритмов по ряду признаков [18]. В зависимости от базовой структуры различают алгоритмы линейные, разветвляющиеся и циклические.

Алгоритм называйся *линейным*, если все действия выполняются однократно в строго определенной последовательности (базовая структура следование).

Алгоритм называется *разветвляющимся* (ветвящимся), если в нем существует выбор одной из нескольких альтернатив последовательных действий в зависимости от исходных или промежуточных данных (базовая структура ветвления). При ветвлении происходит однократный проход по одной из ветвей решения задачи. Признаком ветвящегося алгоритма является наличие операций условного перехода, когда происходит проверка истинности некоторого логического выражения, и в зависимости от истинности или ложности проверяемого условия выбирается соответствующая ветвь для дальнейшей реализации алгоритма.

Алгоритм называется *циклическим*, если при каждом исполнении предписывается многократное выполнение одной и той же последовательности действий – тела цикла (базовая структура цикл).

Существенная особенность перечисленных базовых структур состоит в том, что каждая из них имеет один вход и один выход. Их можно соединять друг с другом в любой последовательности. В качестве действия может использоваться любая из перечисленных структур, что обеспечивает возможность вложенности одних структур в другие. Возврат назад выполняется только в циклах.

Следует отметить, что по способу исполнения выделяют вспомогательные и параллельные алгоритмы:

*Вспомогательные* алгоритмы – алгоритмы, целиком используемые в составе других алгоритмов. Вспомогательный алгоритм представляет алгоритм решения некоторой подзадачи из исходной (основной) задачи. Вспомогательный алгоритм, записанный на языке программирования, называется процедурой или подпрограммой.

Алгоритм может содержать обращение к самому себе как вспомогательному и в этом случае он называется *рекурсивным*.

*Параллельные* алгоритмы – алгоритмы, предназначенные для вычислительных машин, способных выполнять несколько операций одновременно.

В отдельные классы выделяют имитирующие и эмпирические алгоритмы.

*Имитирующие* алгоритмы создаются на основе описаний действий объектов, наблюдений, зависимости исходных данных от изменяющихся условий.

*Эмпирические* алгоритмы – это алгоритмы, основанные на опыте, изучении фактов и опирающиеся па непосредственное наблюдение и эксперимент.

Выделяют также случайные и самоизменяющиеся алгоритмы.

Алгоритм называется *случайным*, если в процессе его выполнения предусматривается возможность случайного выбора отдельных действий.

Алгоритм называется *самоизмепяющимся*, если он не только перерабатывает входные слова, но и сам изменяется в процессе такой переработки.

*Эвристические* алгоритмы – это алгоритмы теоретически не обоснованные, но позволяющие сократить количество переборов в пространстве поиска. Отметим, что универсальной структуры описания эвристических алгоритмов не существует.

На практике наиболее употребительна классификация по типу решаемых задач.

1. Вычислительный алгоритм (обработка некоторой совокупности возможных исходных данных и получение результата).

2. Логический алгоритм (проверка условия).

3. Моделирующий алгоритм (алгоритм создания и заданного функционирования математической модели). Данный тип алгоритмов используется для описания поведения модели объекта.

4. Адаптивный алгоритм (обладает способностью настраиваться на решаемую задачу).

5. Вероятностный алгоритм (использует случайные данные, результат его так же в каком-то смысле случайный).

6. Алгоритм формирования и функционирования объекта, объектно-ориентированное программирование. Описывает объект какого-то класса. От моделирующего отличается тем, что объект реальный (в моделирующем алгоритме) и объект класса различны по своей природе. Реальный объект - это объект, существующий в природе, для которого создаётся математическая модель и затем данная модель и её функционирование реализуются с помощью алгоритма. Объект класса – это просто структура данных.

## 1.3. Основные методы разработки алгоритмов и алгоритмических структур

К настоящему времени специалисты по вычислительной технике разработали ряд эффективных методов, которые нередко позволяют получать эффективные алгоритмы решения больших классов задач [20]. Пытаясь разработать алгоритм решения той или иной задачи, часто бывает полезно задать вопрос: Какой тип

решения позволяет получить метод декомпозиции, динамическое программирование, «жадный» метод или какой-либо другой стандартный метод?

Следует, однако, подчеркнуть, что существуют задачи, например NP-полные задачи, для которых эти и любые другие известные методы не обеспечивают эффективных решений. Когда встречается подобная задача, нередко бывает полезно определить, обладают ли входные данные для этой задачи какими-либо особыми характеристиками, которыми можно воспользоваться при попытках найти требуемое решение, и можно ли воспользоваться каким-либо приближенным решением (если такое решение легко получить) вместо точного решения, получить которое бывает очень трудно.

I. Алгоритмы «Разделяй и властвуй»

Возможно, самым важным и наиболее широко применимым методом проектирования эффективных алгоритмов является метод, называемый методом декомпозиции (или метод «разделяй и властвуй», или метод разбиения). Этот метод предполагает такую декомпозицию (разбиение) задачи размера $n$ на более мелкие задачи, что на основе решений этих более мелких задач можно легко получить решение исходной задачи. Существует множество применений этого метода, например в сортировке слиянием или в деревьях двоичного поиска.

Чтобы проиллюстрировать этот метод, рассмотрим хорошо известную головоломку «Ханойские башни». Имеются три стержня $A$, $B$ и $C$. Вначале на стержень $A$ нанизаны несколько дисков: диск наибольшего диаметра находится внизу, а выше – диски последовательно уменьшающегося диаметра, как показано на рис. 7. Цель головоломки – перемещать диски (по одному) со стержня на стержень так, чтобы диск большего диаметра никогда не размещался выше диска меньшего диаметра и чтобы в конце концов все диски оказались нанизанными на стержень $B$. Стержень $C$ можно использовать для временного хранения дисков.

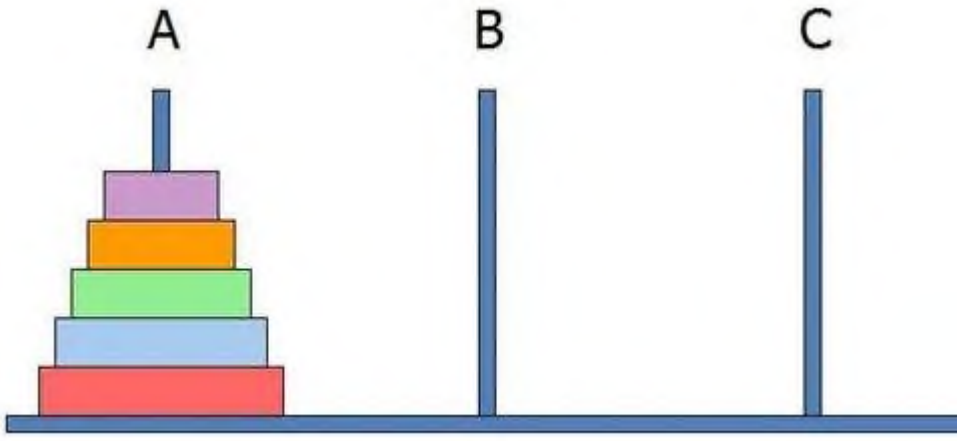



*Рисунок 1. Ханойские башни*

Для решения этой головоломки подходит следующий простой алгоритм. Представьте, что стержни являются вершинами треугольника. Пронумеруем все

перемещения дисков. Тогда при перемещениях с нечетными номерами наименьший диск нужно перемещать в треугольнике на соседний стержень по часовой стрелке. При перемещениях с четными номерами выполняются другие допустимые перемещения, не связанные с наименьшим диском.

Описанный выше алгоритм, конечно, правильный и, к тому же, весьма лаконичный. правда, нелегко понять, почему он «работает», да и вряд ли такой алгоритм быстро придет вам в голову. Попробуем вместо этого применить метод декомпозиции. Задачу перемещения $n$ наименьших дисков со стержня $A$ па стержень $B$ можно представить себе состоящей из двух подзадач размера $n-1$. Сначала нужно переместить $n-1$ наименьших дисков со стержня $A$ на стержень $C$, оставив на стержне $A$ $n$-й наибольший диск. Затем этот диск нужно переместить с $A$ на $B$. Потом следует переместить $n-1$ дисков со стержня $C$ на стержень $B$. Это перемещение $n-1$ дисков выполняется путем рекурсивного применения указанного метода. Чтобы прояснить идею алгоритма, опишем еще одни шаг решения головоломки. После того как $n-1$ дисков перемешены на стержень $C$, а наибольший диск помешен на стержень $B$. $n-2$ наименьших диска со стержня $C$ перемешаются на стержень $A$ и $(n-1)$-й диск переносится на диск $B$. Далее решается задача с $n-2$ дисками, находящимися на диске.

Поскольку диски, участвующие в перемещениях, по размеру меньше тех, которые в перемещении не участвуют, не нужно задумываться над тем, что находится под перемещаемыми дисками на стержнях $A$, $B$ или $C$. Хотя фактическое перемещение отдельных дисков не столь очевидно, а моделирование вручную выполнить непросто из-за образования стеков рекурсивных вызовов, с концептуальной точки зрения этот алгоритм все же довольно прост для понимания и доказательства его правильности, а если говорить о быстроте разработки, то ему вообще нет равных. Именно легкость разработки алгоритмов по методу декомпозиции обусловила огромную популярность этого метода; к тому же, во многих случаях эти алгоритмы оказываются более эффективными, чем алгоритмы, разработанные традиционными методами.

При проектировании алгоритмов приходится идти на различные компромиссы. Очевидно, что по мере возможности необходимо сбалансировать вычислительные затраты на выполнение различных частей алгоритма.

Аналогично, при использовании алгоритмов декомпозиции желательно, чтобы подзадачи были примерно одинакового размера. Например, сортировку вставками можно рассматривать как разбиение задачи на две подзадачи – одна размером 1, а другая – $n-1$, причем максимальные затраты на выполнение слияния равняются $n$ шагам. Складывается впечатление, что разбиение задачи на равные (или примерно равные) подзадачи является важным фактором обеспечения высокой эффективности алгоритмов.

II. Динамическое программирование

Нередко не удается разбить задачу на небольшое число подзадач, объединение решений которых позволяет получить решение исходной задачи. В таких случаях мы можем попытаться разделить задачу на столько подзадач, сколько необходимо, затем каждую подзадачу разделить на еще более мелкие подзадачи и т.д. Если бы весь алгоритм сводился именно к такой последовательности действий, мы бы получили в результате алгоритм с экспоненциальным временем выполнения.

По зачастую удается получить лишь полиномиальное число подзадач и поэтому ту или иную подзадачу приходится решать многократно. Если бы вместо этого мы отслеживали решения каждой решенной подзадачи и просто отыскивали в случае необходимости соответствующее решение, мы бы получили алгоритм с полиномиальным временем выполнения.

С точки зрения реализации иногда бывает проще создать таблицу решений всех подзадач, которые нам когда-либо придется решать. Мы заполняем эту таблицу независимо от того, нужна ли нам на самом деле конкретная подзадача для получения общего решения. Заполнение таблицы подзадач для получения решения определенной задачи получило название динамического программирования (это название происходит из теории управления).

Динамическим программированием (в наиболее общей форме) называют процесс пошагового решения задач, когда на каждом шаге выбирается одно решение из множества допустимых (на этом шаге) решений, причем такое, которое оптимизирует заданную целевую функцию или функцию критерия. В основе теории динамического программирования лежит принцип оптимальности Беллмана.

Формы алгоритма динамического программирования могут быть разными – обшей их «темой» является лишь заполняемая таблица и порядок заполнения се элементов.

Допустим, две команды $A$ и $B$ проводят между собой турнир. Победителем считается тот, кто первым выиграет $n$ матчей. Можно предположить, что силы команд $A$ и $B$ примерно равны, т.е. у каждой из них есть $50\%$ шансов выиграть очередной матч. Допустим, $P(i,j)$ – вероятность того, что если $A$ для победы нужно провести $i$ матчей, а $B-j$ матчей, то в конечном счете победу на турнире одержит $A$. Например, если в турнире «Мировой серии» команда Dodgers выиграла два матча, а команда Yankees – один, то $i=2, j=3$ и оказывается, что $P(2,3)$ равняется 11/16.

Чтобы вычислить $P(i,j)$, можно воспользоваться рекуррентным уравнением с двумя переменными. Во-первых, если $i=0$ и $j>0$, то команда $A$ уже выиграла турнир, поэтому $P(0,j)=1$. Аналогично, $P(i,0)=0$ для $i>0$. Если как $i$ , так и $j$ больше нуля, командам придется провести по крайней мере еще один матч с равными шансами выиграть эту игру. Таким образом, $P(i,j)$ должно быть средним значением $P(i-1,j)$ и $P(l,j-1)$, первое из этих выражений представляет собой вероятность того, что команда $A$ выиграет турнир, если победит в следующем матче, а второе -

вероятность того, что команда $A$ выиграет турнир, даже если проиграет следующий матч. Итак, получаем следующие рекуррентные соотношения:

$$P(i,j) = 1, \text{если } i = 0 \text{ и } j > 0,$$
$$P(i,j) = 0, \text{если } i > 0 \text{ и } j = 0,$$
$$P(i,j) = \big(P(i-1,j) + P(i,j-1)\big)/2, \text{если } i > 0 \text{ и } j > 0.$$

На основании этих соотношений можно показать, что вычисление $P(i,j)$ требует времени не больше, чем $O\big(2^{i+j}\big)$. Обозначим через $T(n)$ максимальное время, которое требуется для вычисления $P(i,j)$, где $i + j = n$. Тогда

$$T(1) = c,$$
$$T(n) = 2T(n-1) + d,$$

для некоторых констант $c$ и $d$. Можно проверить, что $T(n) < 2^{n-1}c + (2^{n-1} - 1)d$, что соответствует $O(2^n)$ или $O\big(2^{i+j}\big)$.

Таким образом, мы установили экспоненциальную верхнюю границу времени, требуемого для рекурсивных вычислений $P(i,j)$. Но если мы хотим убедиться в том, что рекуррентная формула для $P(i,j)$ - не самый лучший способ вычисления этой вероятности, надо найти нижнюю границу времени вычисления. В качестве упражнения рекомендуем читателям показать, что при вычислении $P(i,j)$ общее количество вычислений вероятностей $P$ с меньшими значениями $i$ и $j$, которые приходится выполнять, равняется $C_{i+j}^{i}$, т.е. числу способов выбрать $i$ элементов из $i + j$. Если $i - j$, это число равно $\Omega\big(2^n/\sqrt{n}\big)$, где $n = i + j$. Таким образом, $T(n)$ равняется $\Omega\big(2^n/\sqrt{n}\big)$, и, по существу, мы можем показать, что оно также равняется $O\big(2^n/\sqrt{n}\big)\cdot$ В то время как $2^n/\sqrt{n}$ растет медленнее, чем $2^n$, разница все же невелика, и $T(n)$ растет настолько быстро, что использование рекуррентных вычислений $P(i,j)$ является совершенно неоправданным.

Проблема с рекуррентными вычислениями заключается в том, что нам приходится периодически вычислять одно и то же значение $P(i,j)$ несколько раз. Если, например, надо вычислить $P(2,3)$, то требуется сначала вычислить $P(1,3)$ и $P(2,2)$. Однако для вычисления $P(1,3)$ и $P(2,2)$ нужно вычислить $P(1,2)$, т.е. одно и то же значение $P(1,2)$ мы вычисляем дважды.

Более рациональным способом вычисления $P(i,j)$ является заполнение специальной таблицы, в нижней строке которой присутствуют одни нули, а крайний правый столбец целиком заполнен единицами (это соответствует первым двум строкам).

| | | | | | | |
|---|---|---|---|---|---|---|
| 1/2 | 21/32 | 13/16 | 15/16 | 1 | 4 | |
| 11/32 | 1/2 | 11/16 | 7/8 | 1 | 3 | ↑ |
| 3/16 | 5/16 | 1/2 | 3/4 | 1 | 2 | $j$ |
| 1/16 | 1/8 | 1/4 | 1/2 | 1 | 1 | |
| 0 | 0 | 0 | 0 | | 0 | |
| 4 | 3 | 2 | 1 | 0 | | |

$\leftarrow i$

Что же касается последней строки, то все остальные элементы таблицы представляют собой средние значения элементов, находящихся снизу и справа от соответствующего элемента. Таким образом, таблицу следует заполнять начиная с ее нижнего правого постоянным значением $i + j$.

III. Жадные алгоритмы

Рассмотрим небольшую «детскую» задачу. Допустим, что у нас есть монеты достоинством 25, 10. 5 копеек и 1 копейка и нужно вернуть сдачу 63 копейки. Почти не раздумывая, мы преобразуем эту величину в две монеты по 25 копеек, одну монету в 10 копеек и три монеты по одной копейке. Нам не только удалось быстро определить перечень монет нужного достоинства, но и, по сути, мы составили самый короткий список монет требуемого достоинства.

Алгоритм. которым читатель в этом случае наверняка воспользовался, заключался в выборе монеты самого большого достоинства (25 копеек), но не больше 63 копеек, добавлению се в список сдачи и вычитанию ее стоимости из 63 (получается 38 копеек). Затем снопа выбираем монету самого большого достоинства, но не больше остатка (38 копеек): этой монетой опять оказывается монета в 25 копеек. Эту монету мы опять добавляем в список сдачи, вычитаем ее стоимость из остатка и т.д.

Этот метод внесения изменений называется «жадным» алгоритмом. На каждой отдельной стадии «жадный» алгоритм выбирает тот вариант, который является локально оптимальным в том или ином смысле. Обратите внимание, что алгоритм для определения сдачи обеспечивает в целом оптимальное решение лишь вследствие особых свойств монет. Если бы у нас были монеты достоинством 1 копейка, 5 и 11 копеек и нужно было бы дать сдачу 15 копеек, то "жадный" алгоритм выбрал бы сначала монету достоинством 11 копеек, а затем четыре монеты по одной копейке, т.е. всего пять монет. Однако в данном случае можно было бы обойтись тремя монетами по 5 копеек.

Примерами жадных алгоритмов являются алгоритм построения кратчайшего пути Дейкстры и алгоритм построения остовного дерева минимальной стоимостью Крускала. Алгоритм кратчайшего пути Дейкстры является «жадным» в том смысле, что он всегда выбирает вершину, ближайшую к источнику, среди тех, кратчайший путь которых еще неизвестен. Алгоритм Крускала также «жадный»; он выбирает из остающихся ребер, которые не создают цикл, ребро с минимальной стоимостью.

Следует подчеркнуть, что не каждый «жадный» алгоритм позволяет получить оптимальный результат в целом. Как нередко бывает в жизни, «жадная стратегия» подчас обеспечивает лишь сиюминутную выгоду, в то время как в целом результат может оказаться неблагоприятным. Посмотрим, например, что произойдет, если в алгоритмах Дейкстры и Крускала допустить наличие ребер с отрицательными весами. Оказывается, что на алгоритм Крускала построения остовного дерева это

никак не повлияет: с его помощью по-прежнему можно будет получить дерево минимальной стоимости. Но алгоритм Дейкстры в некоторых случаях уже не позволяет получить кратчайшие пути.

На рис. 8 показан граф с ребром отрицательной стоимости между вершинами $b$ и $c$. Если источником является вершина $s$, то алгоритм Дейкстры сначала правильно определяет, что минимальный путь до $a$ имеет протяженность 1. Теперь, рассматривая ребра от $s$ (или $a$) до $b$ или $c$, алгоритм рассчитывает, что кратчайший путь от $s$ до $b$. а именно $s \to a \to b$, имеет длину 3. Но далее получаем, что с имеет кратчайший путь от $s$ длиной 1.

Однако «жадный» выбор $b$ вместо с является неоправданным. Оказывается, что путь $s \to a \to c \to b$ имеет длину лишь 2, поэтому вычисленное нами минимальное расстояние для $b$ является неправильным.

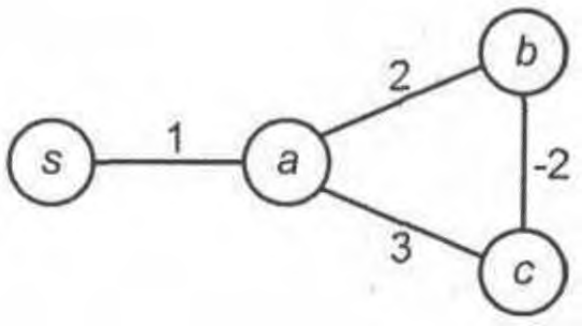



*Рисунок 2. Граф с ребром отрицательного веса*

Вообще говоря, когда допускается наличие ребер с отрицательными весами (стоимостью), к понятию «кратчайший путь» нужно относиться очень осторожно. Если, например, допускается наличие циклов с отрицательной стоимостью, то ничто не мешает многократно проходить такой цикл, получая при этом сколь угодно большие отрицательные расстояния. Поэтому было бы вполне естественным ограничиться лишь ациклическими путями.

Существуют задачи, для которых ни один из известных «жадных» алгоритмов не позволяет получить оптимального решения; тем не менее имеются «жадные» алгоритмы, которые с большой вероятностью позволяют получать «хорошие» решения. Нередко вполне удовлетворительным можно считать «почти оптимальное» решение, характеризующееся стоимостью, которая лишь на несколько процентов превышает оптимальную. В таких случаях «жадный» алгоритм зачастую оказывается самым быстрым способом получить «хорошее» решение. Вообще говоря, если рассматриваемая задача такова, что единственным способом получить оптимальное решение является использование метода полного поиска, тогда «жадный» алгоритм или другой эвристический метод получения хорошего (хотя и необязательно

оптимального) решения может оказаться единственным реальным средством достижения результата.

IV. Поиск с возвратом

Иногда приходится иметь дело с задачей поиска оптимального решения, когда невозможно применить ни один из известных методов, способных помочь отыскать оптимальный вариант решения, и остается прибегнуть к последнему средству – полному перебору.

Рассмотрим какую-либо игру с участием двух игроков, например шахматы, шашки или «крестики-нолики». Игроки попеременно делают ходы, и состояние игры отражается соответствующим положением на доске. Допустим, что есть конечное число позиций на доске и в игре предусмотрено определенное «правило остановки», являющееся критерием ее завершения. С каждой такой игрой можно ассоциировать дерево, называемое деревом игры. Каждый узел такого дерева представляет определенную позицию на доске. Начальная позиция соответствует корню дерева. Если позиция $x$ ассоциируется с узлом $n$, тогда потомки узла и соответствуют совокупности допустимых ходов из позиции $x$, и с каждым потомком ассоциируется соответствующая результирующая позиция на доске. Например, на рис. 9 показана часть дерева для игры в «крестики–нолики».

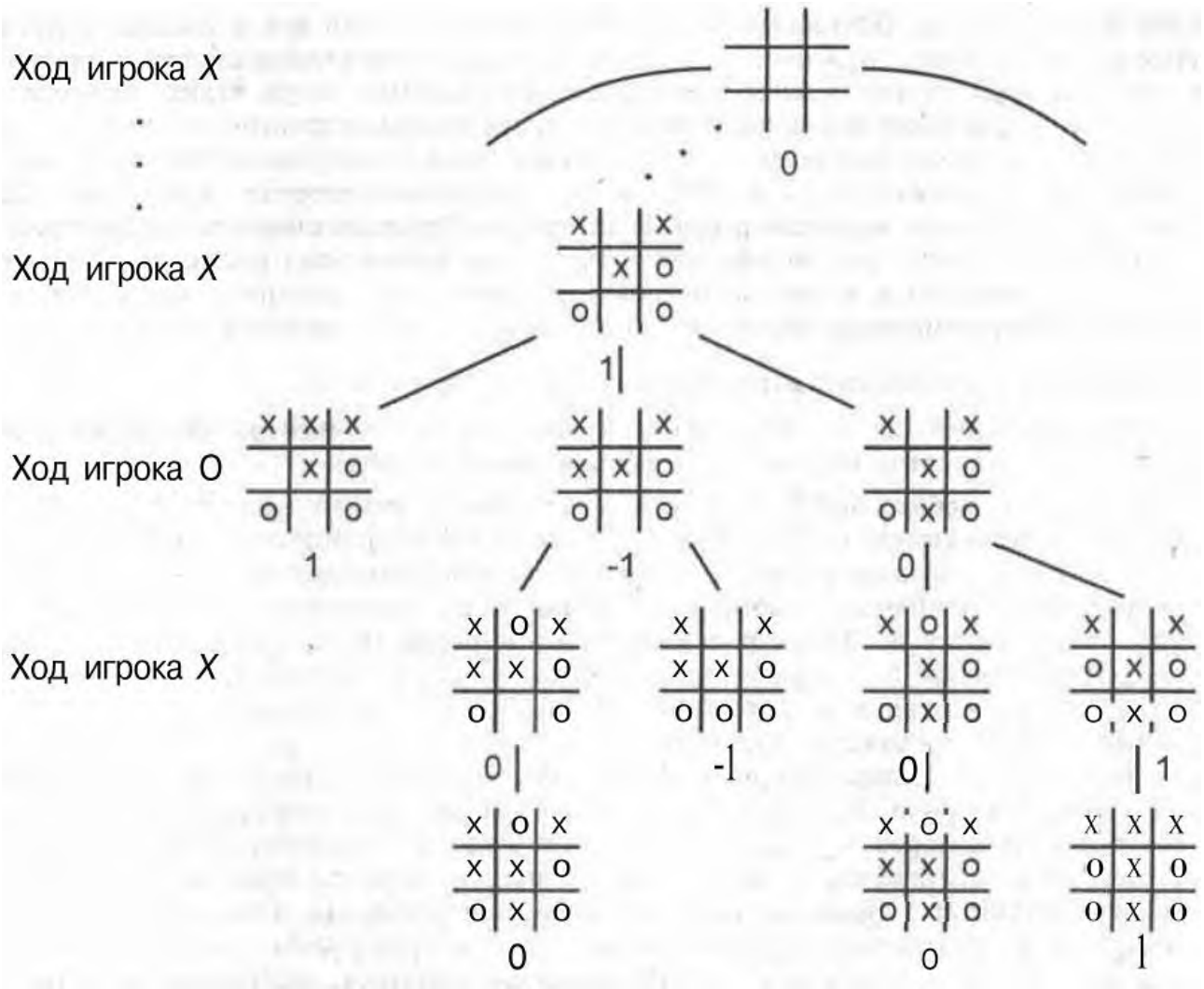



*Рисунок 3. Часть дерева игры в «крестики–нолики»*

Листья этого дерева соответствуют таким позициям на доске, из которых невозможно сделать ход, – то ли потому, что кто-то из игроков уже одержал победу, то ли потому, что все клетки заполнены и игра закончилась вничью. Каждому узлу дерева соответствует определенная цена. Сначала назначается цены листьям. Допустим, речь идет об игре в «крестики–нолики». В таком случае листу назначается цена -1, 0 или 1 в зависимости от того, соответствует ли данной позиции проигрыш, ничья или выигрыш игрока 1 (который ставит «крестики»).

Эти цены распространяются вверх по дереву в соответствии со следующим правилом. Если какой-либо узел соответствует такой позиции, из которой должен сделать ход игрок 1, тогда соответствующая цена является максимальной среди цен потомков данного узла. При этом предполагаем, что игрок 1 сделает самый выгодный для себя ход, т.е. такой, который принесет ему самый ценный результат. Если же узел соответствует ходу игрока 2, тогда соответствующая цена является минимальной среди цен потомков данного узла. При этом мы предполагаем, что игрок 2 также сделает самый выгодный для себя ход, т.е. такой, который при самом благоприятном исходе приведет к проигрышу игрока 1, а при менее предпочтительном исходе – к ничьей.

На рис. 9 показаны цены позиций в игре «крестики–нолики». Листьям, которые приносят выигрыш для игрока 2 (который рисует «нолики»), назначается цена -1; листьям, которые приносят ничью, назначается цена 0; а листьям, которые приносят выигрыш для игрока 1 (который рисует «крестики»), назначается цена +1. Затем продвигаемся вверх по дереву. На уровне 8, где остается только одна пустая клетка, и предстоит ход игроку 1, ценой для неразрешенных позиций является «максимум» из одного потомка на уровне 9.

На уровне 7, где предстоит ход игроку 2, и имеются два варианта решения, в качестве цены для внутреннего узла принимаем минимальную из цен его потомков. Крайняя слева позиция на уровне 7 представляет собой лист и имеет цену 1, поскольку она является выигрышем для 1. Вторая позиция на уровне 7 имеет цену $\min(0,-1)=-1$, тогда как третья имеет цену $\min(0,-1)=0$. Одна позиция, показанная на уровне 6 (на этом уровне предстоит ход игроку 1), имеет цену $\min(1,-1,0)=1$. Это означает, что для обеспечения выигрыша игроку 1 нужно сделать определенный выбор и в этом случае выигрыш последует немедленно.

Обратите внимание: если цена корня равняется 1, то выигрышная стратегия находится в руках игрока 1. Действительно, когда наступает его очередь ходить, он гарантирован, что может выбрать ход, который приведет к позиции, имеющей цену 1. Когда же наступает очередь игрока 2 делать свой ход, ему не остается ничего иного, как выбрать ход, ведущий к все той же позиции с ценой 1, что для него означает проигрыш. Тот факт, что игра считается завершившейся, гарантирует в конечном счете победу первого игрока. Если же цена корня равняется 0, как в случае игры в «крестики–нолики», тогда ни один из игроков не располагает выигрышной

стратегией и может гарантировать себе лишь ничью, если будет играть без ошибок. Если же цена корня равняется -1, то выигрышная стратегия находится в руках игрока 2.

Идею дерева игры, узлы которого имеют цену -1, 0 или 1, можно обобщить на деревья, листьям которых присваиваются любые числа (такое число называется выигрышем), выполняющие роль цены игры. Для оценивания внутренних узлов применяются те же правила: взять максимум среди потомков на тех уровнях, где предстоит сделать ход игроку 1, и минимум среди потомков на тех уровнях, где предстоит сделать ход игроку 2.

В качестве примера, где удобно применить концепцию выигрышей, рассмотрим сложную игру (например, шахматы), в которой дерево игры, являясь, в принципе, конечным, столь огромно, что любые попытки оценить его по метолу «снизу вверх» обречены на неудачу. Любые шахматные программы действуют, в сущности, путем построения для каждой промежуточной позиции на шахматной доске дерева игры. Корнем этого дерева является текущая позиция: корень распространяется вниз на несколько уровней. Точное количество уровней зависит от быстродействия конкретного компьютера. Когда большинство листьев дерева проявляют «неоднозначность» (т.е. не ведут ни к выигрышу, ни к ничьей, ни к проигрышу), каждая программа использует определенную функцию позиций на доске, которая пытается оценить вероятность выигрыша компьютера в этой позиции. Например, на такой оценке в значительной мере сказывается наличие у одной из сторон материального перевеса и такие факторы, как прочность защиты королей. Используя подобную функцию выигрыша, компьютер может оценить вероятность выигрыша после выполнения им каждого из возможных очередных ходов (в предположении, что игра каждой из сторон будет проходить по оптимальному сценарию) и выбрать ход с наивысшим выигрышем.

Есть ряд других правил, в соответствии с которыми действуют хорошие шахматные программы:

1. Использование эвристик, чтобы исключить из рассмотрения определенные ходы, которые вряд ли можно считать хорошими. Это помогает увеличить количество просматриваемых уровней дерева за фиксированное время.

2. Распространение «цепочек взятия», которые представляют собой последовательности ходов, сопровождающиеся взятием фигур противника, за пределы последнего уровня, до которого обычно распространяется дерево. Это помогает более точно оценить относительную материальную силу позиций.

3. Сокращение поиска на дереве методом альфа-бета отсечения.

V. Алгоритмы локального поиска

Описанная ниже стратегия нередко приводит к оптимальному решению задачи.

1. Начните с произвольного решения.

2. Для улучшения текущего решения примените к нему какое-либо преобразование из некоторой заданной совокупности преобразований. Это улучшенное решение становится новым «текущим» решением.

3. Повторяйте указанную процедуру до тех пор, пока ни одно из преобразований в заданной их совокупности не позволит улучшить текущее решение.

Результирующее решение может, хотя и необязательно, оказаться оптимальным. В принципе, если «заданная совокупность преобразований» включает все преобразования, которые берут в качестве исходного одно решение и заменяют его каким-либо другим, процесс «улучшений» не закончится до тех пор, пока мы не получим оптимальное решение. Но в таком случае время выполнения пункта 2 окажется таким же, как и время, требующееся для анализа всех решений, поэтому описываемый подход в целом окажется достаточно бессмысленным.

Этот метод имеет смысл лишь в том случае, когда мы можем ограничить нашу совокупность преобразований небольшим се подмножеством, что даст возможность выполнить все преобразования за относительно короткое время. Если совокупность преобразований невелика, естественно рассматривать решения, которые можно преобразовывать одно в другое за один шаг, как «близкие». Такие преобразования называются «локальными», а соответствующий метод называется локальным поиском.

Одной из задач, которую можно решить именно методом локального поиска, является задача нахождения минимального остовного дерева. Локальными преобразованиями являются такие преобразования, в ходе которых мы берем то или иное ребро, не относящееся к текущему остовному дереву, добавляем его в это дерево (в результате мы должны получить цикл), а затем убираем из этого цикла в точности одно ребро (предположительно, ребро с наивысшей стоимостью), чтобы образовать новое дерево.

Рассмотрим, например, граф на рис. 10. Мы можем начать с дерева, показанного на рис. 10 а. Одно из преобразований, которые можно было бы выполнить, заключается в добавлении ребра $(d, \mathrm{e})$ и устранении из образовавшегося цикла $(\mathrm{e}, \mathrm{a}, \mathrm{c}, d, \mathrm{e})$ какого-либо другого ребра. Если мы удалим ребро $(\mathrm{a}, \mathrm{e})$, то уменьшим стоимость дерева с 20 до 19. Это преобразование можно выполнить, получив в результате дерево (рис. 10 б), к которому мы опять попытаемся применить улучшающее преобразование. Одно из таких преобразований сводится к вставке ребра $(a, d)$ и удалению из образовавшегося цикла ребра $(\mathrm{c}, d)$. Результат этого преобразования показан на рис. 10 в. Затем можно вставить ребро $(\mathrm{a}, b)$ и убрать ребро $(b, c)$, как показано на рис. 10 г, а потом – вставить ребро $(b, e)$ вместо ребра $(d, \mathrm{e})$. Результирующее дерево (рис. 10 д) является минимальным. Мы можем убедиться в том, что каждое ребро, не входящее в состав этого дерева, имеет наивысшую стоимость среди всех ребер в цикле, который они с этим ребром составляют. Таким образом, к дереву на рис. 10 г преобразования уже неприменимы.

Время, которое занимает выполнение алгоритма на графе из $n$ узлов и $e$ ребер, зависит от количества требующихся улучшений решения. Одна лишь проверка того факта, что преобразования уже неприменимы, может занять $O(ne)$ времени, поскольку для этого необходимо перебрать е ребер, а каждое из них может образовать цикл длиной примерно $n$. Таким образом, этот алгоритм несколько хуже, чем алгоритмы Прима и Крускала, однако он может служить примером получения оптимального решения на основе локального поиска.

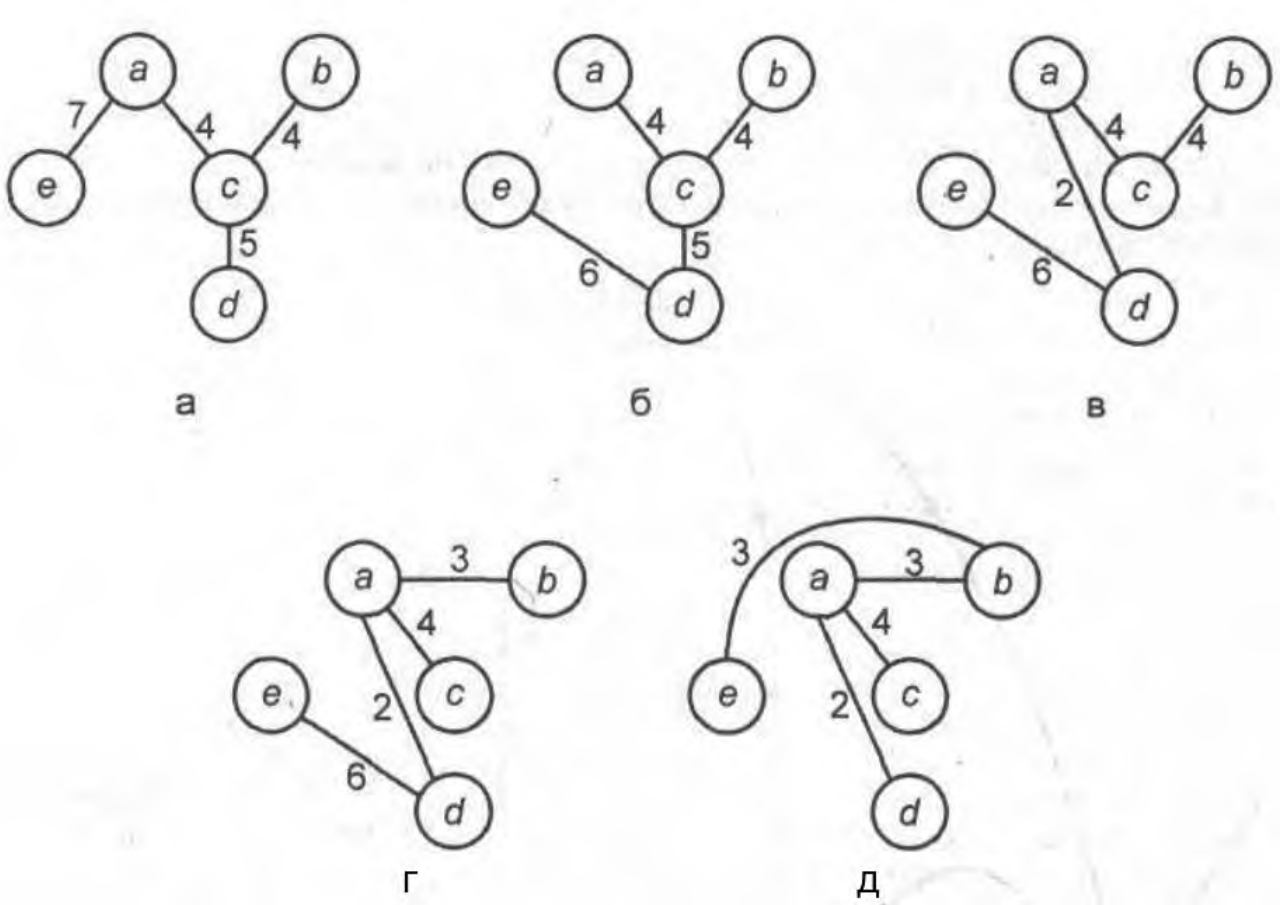



*Рисунок 4. Часть дерева игры в «крестики–нолики»*

VI. Рандомизированные алгоритмы

Во многих практических задачах механический перенос на сложные системы обычных детерминированных алгоритмов, работоспособных для простых систем, приводит к неразрешимым противоречиям [21]. В частности, возникает огромное количество так называемых неразрешимых за реальное время задач. В последнее время все чаще используются «рандомизированные» подходы к решению таких сложных задач.

Рандомизированный алгоритм – это не что иное, как процедура, в которой один или несколько шагов основаны на случайном выборе правила, т.е. при использовании рандомизированных алгоритмов на каком-то этапе вместо того, чтобы самим принять решение, мы призываем судьбу (случай) выбирать за нас. Но тогда естественно возникает вопрос: зачем мудрому человеку прибегать к судьбе? Судьба не является специалистом ни в чем, выбор делается случайно. Итак, почему от

рандомизированных алгоритмов может быть польза? Успешное использование рандомизированных методов требует ясного ответа на этот вопрос.

В последнее время активно развиваются рандомизированные методы решения различных практических задач. С одной стороны, в задачах, требующих большого объема «перебора» вариантов, алгоритмы, основанные на случайном выборе, позволяют за ограниченное время добиваться хороших результатов с определенной вероятностью. С другой стороны, возможность рандомизации процессов наблюдений позволяет во многих случаях компенсировать негативное влияние систематических погрешностей (ошибок моделей).

Традиционный подход к процессу конструирования самых разнообразных систем обработки данных и принятия решений основывается па детерминированных алгоритмах. Этот подход, однако, может быть обобщен включением рандомизации. В рандомизированных алгоритмах один или несколько шагов основываются на случайном выборе, при котором среди многих детерминированных правил выбирается одно в соответствии с некоторой случайной схемой. Рандомизация позволяет ввести новый термин «вероятностно успешный алгоритм». Во многих случаях, когда невозможно достичь детерминированной успешности, вероятностная успешность предлагается как значимая альтернатива. Рандомизация становится мощным средством для решения целого ряда задач, считающихся неразрешимыми с помощью детерминированных методов.

Развитие средств контроля и вычислительной техники в настоящее время позволяет перейти к решению многих практических задач «на лету», встраивая «умные» блоки в контуры управления простых и сложных систем, в технологические процессы, в разнообразные системы поддержки принятия решений и т. д.

Обычно в условиях реального времени для «извлечения» нужной информации мы имеем:

существенные ограничения в ресурсах;

недостаточное количество данных с необходимым разнообразием.

Какие в связи с этим возникают новые трудности при обработке данных и в какой степени возможно их преодолеть?

Точное решение любой задачи возможно при ее точной постановке. Но связи и отношения в реально существующем мире настолько сложны и многообразны, что практически невозможно математически строго описать многие явления. Типичным теоретическим подходом является выбор близкой к реальным процессам математической модели и включение в нее различных помех, частично компенсирующих «грубость» математической модели и характеризующих неконтролируемые внешние возмущения. Трудности в использовании детерминированных подходов приводят к необходимости поиска алгоритмов, обеспечивающих высокое качество оценки при минимальных предположениях о статистических свойствах помех.

Многие авторы выступают за использование рандомизированных алгоритмов для уменьшения сложности вычислений по сравнению с более традиционными детерминированными подходами. Идея состоит в следующем. Если детерминированный алгоритм требует чрезмерных вычислений для обработки всей доступной информации, то можно сознательно отказаться от части информации и перейти к решению упрощенной задачи с частичной информацией.

При этом можно применить рандомизированный подход к определению решения с высокой вероятностью успеха. Конечный результат – компромисс между гарантией полного успеха и вычислительной реализуемостью (возможностью получить ответ за реальное ограниченное время).

Заметим, что причина обращения к рандомизированным методам осталась та же – это недостаток информации. Однако здесь другая подоплека: информацию вполне возможно собрать и всю, но ее часть сознательно отбрасывается для увеличения вычислительной реализуемости.

Успеху рандомизированных алгоритмов способствует еще и тот факт, что производительность и вероятность успеха рандомизированных алгоритмов может быть достаточно точно оценена аналитическими методами.

Истоки рандомизированных алгоритмов лежат в идеях *метода статистического моделирования Монте-Карло* (МК), предложенного в Лос-Аламосе Джоном фон Нейманом, Николасом Метрополисом и Станиславом Уламом в ходе работ над Манхэттенским проектом. Метод МК задает простую схему для оценки среднего значения (математического ожидания) $E\{F(X)\}$ на основе случайной выборки образцов $x_k$ случайной величины $X$ (рандомизации $X$ ). Первоначально стохастический подход использовали для аппроксимации многомерных интегралов в уравнениях переноса, возникших в связи с задачей о движении нейтрона в изотропной среде. В университетских курсах математики обычно приводят следующий пример способа вычисления интеграла:

$$f(\theta) = \int_{x \in X} F(\theta, x) dx$$

которым удобно пользоваться при невозможности аналитического интегрирования. Пусть множество $X$ является параллелепипедом с объемом $V(X)$ и предположим, что значения функции $F$ ограничены: $0 \le F(\theta, x) \le F_{\max}$. Геометрический смысл интегралаобъем под графиком функции $F$. Зафиксируем натуральное число $N$, выберем в параллелепипеде $\Pi \times [0, F_{\max}] \subset R^{d+1}$ случайно $N$ точек $z_1, z_2, \dots, z_N$ и подсчитаем среди них число $K$ тех, у которых последняя координата не превосходит значения функции $F$ в соответствующей точке, определяемой первыми $d$ координатами. Хорошей оценкой для значения интеграла будет:

$$f(\theta) \approx V(X) F_{\max} \frac{K}{N}.$$

Причем, в силу закона больших чисел и неравенства Чебышева, необходимую точность приближения можно гарантировать априори при выборе количества точек

$N$. Более точно, для любого сколь угодно малого $\epsilon > 0$ справедлива следующая оценка для вероятности ошибки:

$$\mathrm{P}\left\{\left|\frac{f(\theta)}{V(X)F_{\max}} - \frac{K}{N}\right| \geq \epsilon\right\} \leq \frac{f(\theta)(V(X)F_{\max} - f(\theta))}{N\epsilon^2(V(X)F_{\max})^2}.$$

При небольшой размерности $d$ множества $X$ во многих случаях можно при вычислении интегралов пользоваться и традиционными детерминированными приближенными методами, но при $d \gg 1$ на практике фактически ничем нельзя воспользоваться, кроме метода Монте-Карло, сложность которого (как видно из последней формулы) не зависит от размерности $d$. И при небольших $d$, когда функция задана неявно, а необходимо определить область, заданную в виде сложных неравенств, стохастический метод может оказаться более предпочтительным.

На рис. 11 приведен пример интегрирования одномерной функции методом Монте-Карло. Для этого 30 случайно выбранных в прямоугольнике $[0; 25] \times [0; 100]$ точек были сравнены с 30 -ю соответствующими значениями функции и было подсчитано количество точек $K = 14$, попавших под график. В результате получаем $f \approx 25 \cdot 100 \cdot 14/30 = 1166\frac{2}{3}$. При этом вероятность того, что относительная ошибка превысит $5\%$ равна примерно 0,11 . Конечно в одномерном случае $(d = 1)$ по 30 измерениям можно было бы хорошо аппроксимировать интеграл и методом разбиения $X$ на 30 отрезков, но при $d \gg 1$, разбивая $X$ на отрезки, ту же точность можно обеспечить при $30^d$ измерений, что несоизмеримо велико по сравнению всего с $N = 30$ вычислений для метода Монте-Карло.

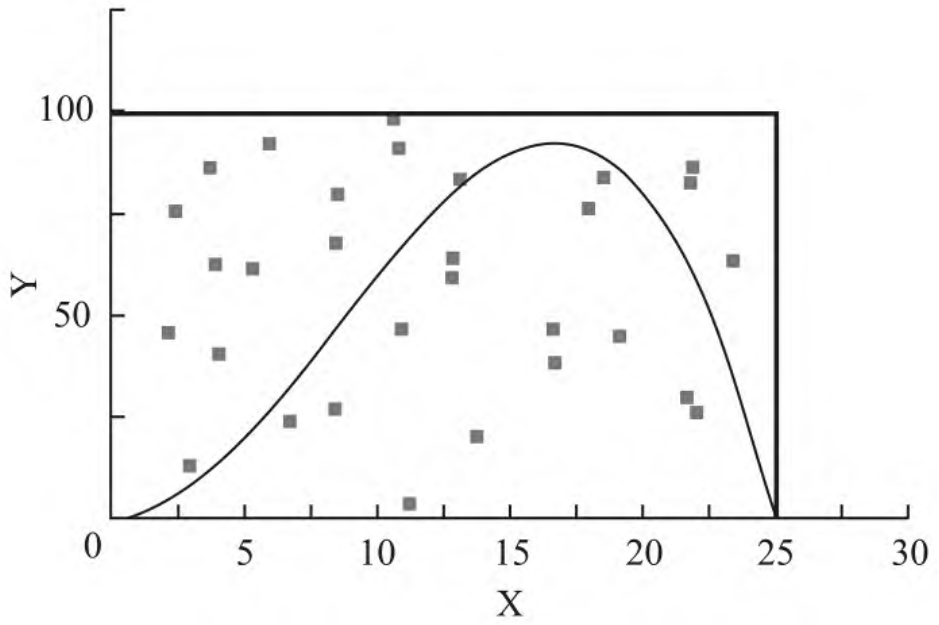



*Рисунок 5. Интегрирование методом Монте-Карло*

Появление и развитие метода Монте-Карло неслучайно совпало по времени с созданием первых вычислительных машин. Первые ЭВМ уже могли с большой скоростью генерировать псевдослучайные числа, что резко расширило круг задач, доя решения которых стохастический подход оказался более эффективным, чем

другие математические методы. В 1950-х годах метод активно использовался для расчетов при разработке водородной бомбы.

Многие практические задачи в математической форме представляются в виде задач о поиске корня некоторой функции или о поиске точки минимума. При этом часто аналитическое решение найти очень трудно. Одон из альтернативных подходов *случайный поиск*, при котором последовательность оценок решения строится итеративно, выбирая на каждой итерации новую оценку на основе сдвига старой в случайно выбираемом направлении. Методы случайного поиска в русскоязычной литературе исследовались, например, в работах Л.А. Растригина, Ю.А. Сушкова, А. Жилинскаса и А.А. Жиглявского. При решении задач глобальной оптимизации в настоящее время активно используются алгоритмы имитации отжига и генетические алгоритмы, в основе которых также лежат рандомизированные правила.

При разработке новых архитектур процессоров в компании Intel активно используется метод случайного поиска в рандомизированном алгоритме расчета пиксельной фазовой маски доя иммерсионной литографии одного из основных этапов технологического процесса производства микросхем.

С. Якушкин успешно использовал рандомизированные алгоритмы случайного поиска при генерации аппаратной и программной частей вычислительных устройств.

Для адаптивной подстройки изменяемого параметра сервера, использующегося для обработки очереди заданий, было предложено использовать рандомизированный алгоритм стохастической аппроксимации.

Рандомизированные алгоритмы стохастической оптимизации используются в системе управления загрузкой узлов распределенной вычислительной сети.

Рандомизированный алгоритм стохастической аппроксимации применяется в задаче самообучения.

В последних четырех примерах традиционные предположения о независимости и центрированности внешних помех не соблюдаются в силу специфики рассматриваемых задач. При этом рандомизация способствовала не только ускорению процесса обработки данных, но и снижению негативного влияния «почти произвольных» внешних помех наблюдений.

Идея использования случайных входных сигналов для устранения эффекта смещения была выдвинута Р. Фишером в виде рандомизированного принципа планирования эксперимента. Рекуррентные алгоритмы оценивания параметров регрессии при случайных входных сигналах рассматривались также в книгах А.Алберта, Л. Лыонга, Т. Сёдерстрёма, П. Юнга, но при стандартных предположениях о помехах, а именно, считалось, что они представляют собой последовательности случайных величин с нулевым средним, независимых или слабозависимых. О.Н. Граничиным, А.В. Гольденшлюгером и Б.Т. Поляком в были предложены рандомизированные алгоритмы оценки параметров линейной регрессии, состоятельные при почти произвольных помехах.

Одна из существенных проблем, с которой сталкиваются *при синтезе схем управления*, это недостаточная вариативность последовательности наблюдений: «управляющие воздействия должны быть в известной мере изучающими, но в известной мере направляющими» (А. А. Фельдбаум). Например, если цель адаптивного управления состоит в минимизации отклонения вектора состояния системы от заданной траектории, то это часто приводит к вырожденной последовательности наблюдений, когда получаемых данных не хватает до я определения важных характеристик системы (ее идентификации). В то время как доя успешного проведения идентификации неизвестных параметров системы должно быть обеспечено «разнообразие» наблюдений. Учитывая возможности прямого влияния на управляемые процессы через входные переменные системы, в 1986 г. О.Н. Граничиим совместно с В. II. Фоминым и Х.-Ф. Ченом с Л. Гао было предложено включать в канал управления пробное рандомизированное возмущение дополнительно к основному управлению при синтезе адаптивного асимптотически-онтимального управления (минимаксного и среднеквадратического соответственно). В современных работах М. Камни, Э. Вейера и Дж. Калафиори, Л. Фаджиапо рассматриваются перспективы использования такого же типа рандомизированных стратегий управления для идентификации и синтеза робастного оптимального управления на конечном интервале времени.

В связи с массовым переходом к обработке потоков двумерных (2-D) и трехмерных (3-D) данных резко увеличились объемы обрабатываемой информации. Сложность традиционных методов квантования сигналов возрастает по экспоненциальному закону с ростом размерности сигналов (звуки, картинки, объемные тела). Квантование 1-D сигналов при $N = 10^3$ отсчетах соответствует объему данных, пропорциональному $10^6$ в случае $2 - \mathrm{D}$, а в случае $3 - \mathrm{D} - 10^9$. В современных приложениях для цифровых фото- и видеокамер требования о необходимой частоте (скорости, Nyquist Rate) измерений настолько высоки, что слишком большое количество получающихся данных надо сильно сжимать перед хранением или пересылкой. В других приложениях, включая системы отображения (медицинские сканеры, радары) и быстродействующие аналого-цифровые конвертеры, увеличение частоты измерений оказывается очень дорогостоящим. С практической точки зрения очень полезно исследовать возможности восстановления многомерных сигналов $\theta \in R^d$ по сравнительно небольшому набору наблюдений $y \in R^m$ при $m < d$, что, конечно же, нереализуемо в общем случае. Но в последнее время на помощь традиционной теории обработки сигналов приходит новая парадигма compressive sensing («опознание со сжатием»), позволяющая достаточно точно восстанавливать «разреженные» (sparse) многомерные сигналы. При этом используются *рандомизированные измерения* по схеме

$$y = A\theta + v,$$

где $\theta$ является восстанавливаемым сигналом, а $A$ – случайная матрица оператора измерений, преобразующFя сигналы в наборы данных размерности $m \times d$. По результатам этих измерений исходная информация восстанавливается почти полностью за счет применения методов $\ell_1$-оптимизации: $\theta$ находится как решение некоторой задачи вида:

$$\| \theta \|_1 = \sum_j |\theta[j]| \to \min: \quad y = \sum_{j=1}^{N} \theta[j]u[j].$$

Одним из важных наблюдений является то, что в рандомизированном алгоритме выбор некоторых действий делается наугад, т. е. для выбора призываем судьбу. Так как «судьба» не является специалистом ни в чем, то преимущество использования рандомизированного алторитма не всегда может быть выражено «хорошим» выбором судьбы. Преимущество в новом восприятии проблемы разрешимости состоит в том, что в дополнение к двум крайним значениям «белое»-«черное» («разрешимо»–«неразрешимо») вводится множество оттенков «серого». С точки зрения перспектив решений для «худшего» случая (worst-case) «серый» результат может быть более приемлемым, чем несколько «черных» среди многих «белых».

Важно отметить, что в процессе оценки уровня «серого» можно прибегать к результатам теории вероятности, чья эффективность порой действительно удивляет. Это добавляет уверенности в перспективах использования рандомизированных методов.

# СПИСОК ЛИТЕРАТУРЫ


1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.